

Machine Learning for Systems and Control

5SC28

Lecture 8

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group

Department of Electrical Engineering

Eindhoven University of Technology

Academic Year: 2025-2026 (version 26/05/2026)

Past Lectures

Introduction to Reinforcement Learning

Temporal Difference Learning

Value-Based Learning: Tabular Q-Learning

Approximate Q-Learning

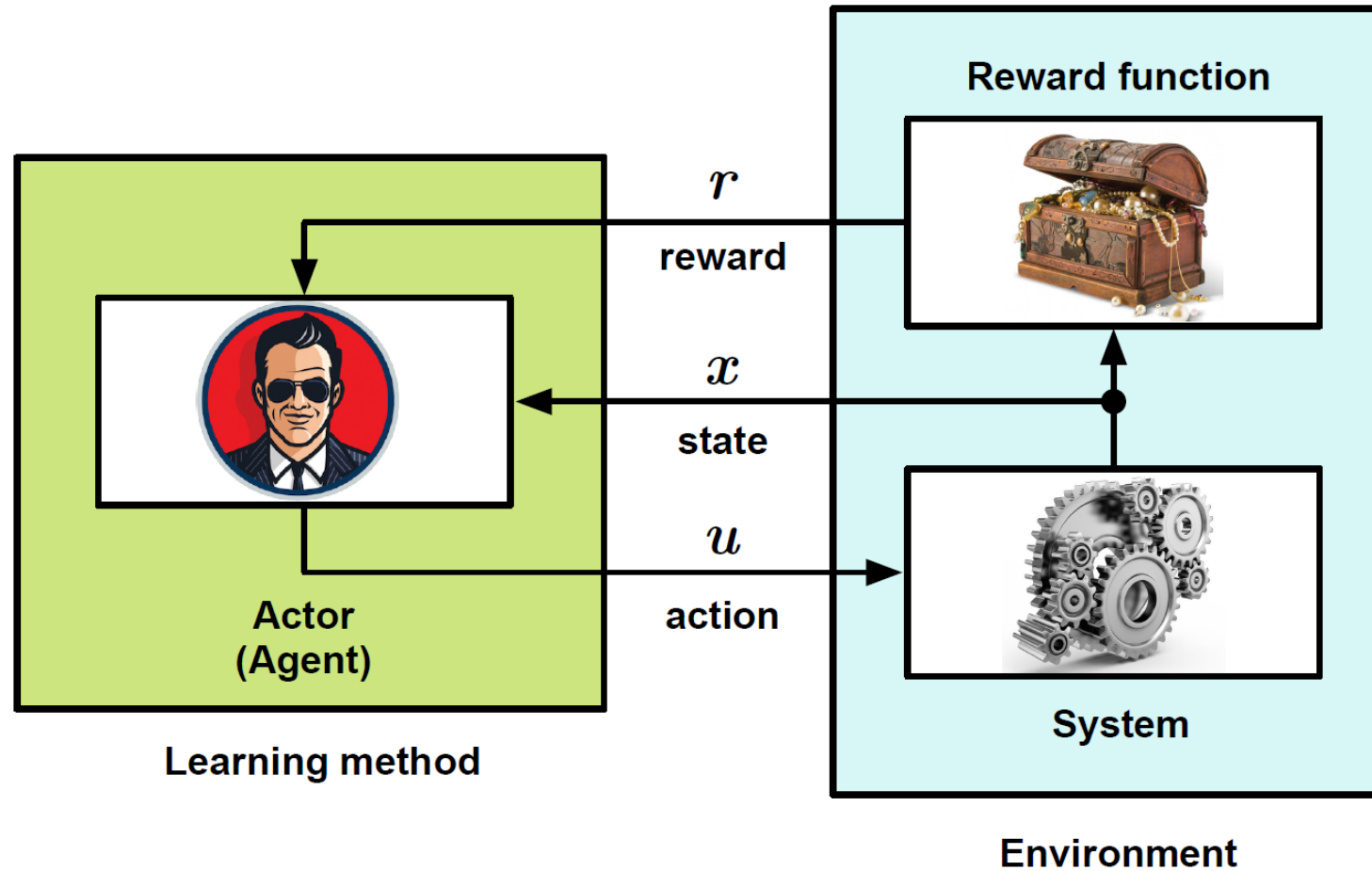
DQN

GP-based Q-Learning

Learning Goals

- **Explain, discuss, compare and interpret** the main techniques and theory for reinforcement learning based control starting from classical Q-learning up to **actor-critic** and model internalization-based methods;
- **Recommend and evaluate** a machine learning method for a real-life application in a systems and **control setting**.
- **Implement, tailor, and apply** the Gaussian process, (deep) neural network and **reinforcement learning techniques** for model and control learning on real-world systems (e.g. on an inverted pendulum laboratory setup).

Action-Value Reinforcement Learning



Agent learns Q_* or Q_π to improve its actions

Action-Value Reinforcement Learning

Concept: Learn the value of actions through $Q_*(x,u)$ or $Q_\pi(x,u)$

Strategy:

Training & Data $\rightarrow$ θ $\rightarrow$ Q-function $\rightarrow$ Policy π

Action-Value Reinforcement Learning

Concept: Learn the value of actions through $Q_*(x,u)$ or $Q_\pi(x,u)$

Strategy:

Training & Data $\rightarrow$ θ $\rightarrow$ Q-function $\rightarrow$ Policy π

Disadvantages: Finite action space to solve $\pi_*(x) = \arg \max_u Q_*(x, u)$
Poor exploration in case of large action space (slow)

Policy Gradient Reinforcement Learning

Concept: Learn the policy that ensures maximum rewards for the actions taken

Strategy: Training & Data $\rightarrow$ θ $\rightarrow$ Policy π

Policy Gradient Reinforcement Learning

Concept: Learn the policy that ensures maximum rewards for the actions taken

Strategy: Training & Data $\rightarrow$ θ $\rightarrow$ Policy π

Main Advantages: No need to solve $\pi_*(x) = \arg \max_u Q_*(x, u)$

No need to discretize the action space

Can handle stochastic policies

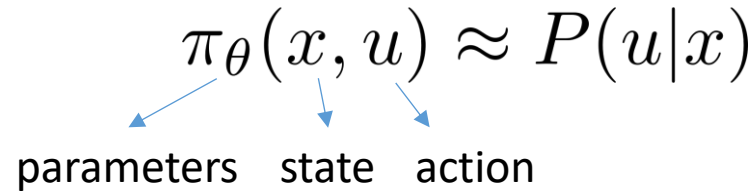
Increased algorithm stability

(Dis)advantage: More complicated algorithm, more options to tune

Policy Gradient Methods

Stochastic Policy Representation

Stochastic policy: Learn a function that gives the probability distribution over actions from the current state:

$$\pi_{\theta}(x, u) \approx P(u|x)$$


parameters state action

Stochastic policies can be optimal in a partially observed or competitive environment (e.g. card games)

Stochastic Policy Representation

Stochastic policy: Learn a function that gives the probability distribution over actions from the current state:

$$\pi_{\theta}(x, u) \approx P(u|x)$$

Discrete actions: softmax policy

$$\pi_{\theta}(x, u) = \frac{\exp(f_{\theta}(x, u))}{\sum_{u'} \exp(f_{\theta}(x, u'))}$$

Continuous actions: Gaussian policy

$$\pi_{\theta}(x, u) = \frac{1}{\sqrt{2\pi}\sigma_{\theta}(x)} \exp\left(-\frac{(u - f_{\theta}(x))^2}{2\sigma_{\theta}(x)^2}\right)$$

Policy Gradient: Optimization

Objective function: $J(\theta, x) = V_{\pi}(x)$

Interpretation: J is the expected cumulative reward (via the value function) using the policy π .

Policy Gradient: Optimization

Objective function: $J(\theta, x) = V_{\pi}(x)$

Interpretation: J is the expected cumulative reward (via the value function) using the policy π .

Solution: Stochastic Gradient **Ascent** (SGA)

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta_k, x_k)$$

Where α is the step size. Replay can be used to assist convergence.

Policy Gradient: Optimization

Objective function: $J(\theta, x) = V_{\pi}(x)$

Interpretation: J is the expected cumulative reward (via the value

function) given the policy π .

How to calculate the value gradient to arrive to a gradient over the policy?

Solution: Stochastic Gradient **Ascent** (SGA)

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta_k, x_k)$$

Where α is the step size. Replay can be used to assist convergence.

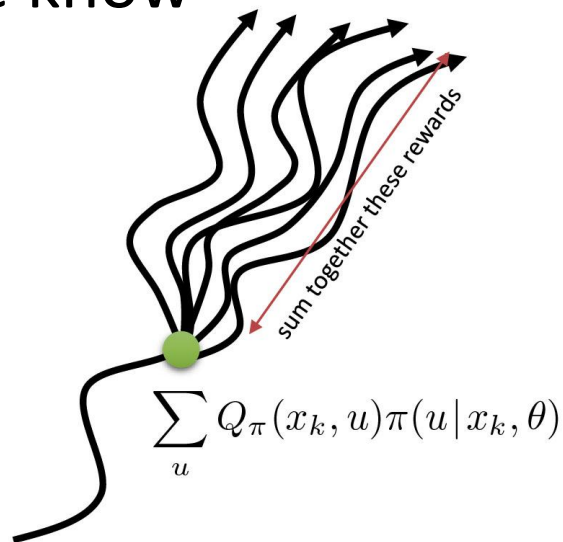
Policy Gradient Theorem

Assumptions:

Assume U is a discrete action space (for simplicity)

Probability that action u is taken for a given x, θ is nonzero over U

We know



$$V_\pi(x) = \sum_u Q_\pi(x, u) \pi(u | x, \theta)$$

i.e., we can characterize the effect of the current action made by the policy over the future based on Q_π

Policy Gradient Theorem

We can show in terms of the policy gradient theorem^{1,2}:

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \nabla_{\theta} \pi(u|x, \theta)$$

Equivalent reformulation:

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \pi(u|x, \theta) \frac{\nabla_{\theta} \pi(u|x, \theta)}{\pi(u|x, \theta)}$$

Or:

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \pi(u|x, \theta) \nabla_{\theta} \ln \pi(u|x, \theta)$$

¹ R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction." A Bradford Book, pp. 356, 2008.

² V. Konda and J. Tsitsiklis, "Actor-Critic Algorithms." SIAM Journal on Control and Optimization, pp. 1008-1014, 2000.

Policy Gradient Theorem

We can show in terms of the policy gradient theorem^{1,2}:

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \nabla_{\theta} \pi(u | x, \theta)$$

Equivalent formulation:

The derivative of the expected reward is the expectation of the product of the reward and gradient of the log of the policy π_{θ} .

Or:

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \pi(u | x, \theta) \nabla_{\theta} \ln \pi(u | x, \theta)$$

¹ R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction." A Bradford Book, pp. 356, 2008.

² V. Konda and J. Tsitsiklis, "Actor-Critic Algorithms." SIAM Journal on Control and Optimization, pp. 1008-1014, 2000.

Policy Gradient Algorithm Backbone

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \pi(u | x, \theta) \nabla_{\theta} \ln \pi(u | x, \theta)$$



Replace u with the sample as the current action taken

$$u_k \sim \pi(x_k, \theta)$$

$$\nabla_{\theta} J(\theta, x_k) \propto \mathbb{E}_{\pi} \{ Q_{\pi}(x_k, u_k) \nabla_{\theta} \ln \pi(u_k | x_k, \theta) \}$$



$$\mathbb{E}_{\pi} \{ G_k | x_k, u_k \} = Q_{\pi}(x_k, u_k)$$

$$\nabla_{\theta} J(\theta, x_k) \propto \mathbb{E}_{\pi} \{ G_k \nabla_{\theta} \ln \pi(u_k | x_k, \theta) \}$$

Policy Gradient Algorithm Backbone

$$\nabla_{\theta} J(\theta, x) \propto \sum_u Q_{\pi}(x, u) \pi(u|x, \theta) \nabla_{\theta} \ln \pi(u|x, \theta)$$



Replace u with the sample as the current action taken

$$u_k \sim \pi(x_k, \theta)$$



Implement in REINFORCE algorithm



$$\mathbb{E}_{\pi} \{G_k \mid x_k, u_k\} = Q_{\pi}(x_k, u_k)$$

$$\nabla_{\theta} J(\theta, x_k) \propto \mathbb{E}_{\pi} \{G_k \nabla_{\theta} \ln \pi(u_k | x_k, \theta)\}$$

REINFORCE Algorithm

Input: a differentiable policy parameterization $\pi(u|x, \theta)$

Parameters: step size $\alpha \in (0, 1]$

Initialize: policy parameters θ

for each episode **do**

 Generate an episode $x_0, u_0, r_1, \dots, x_{T-1}, u_{T-1}, r_T$

repeat for episode step $t = 0, 1, \dots, T$

$$\hat{G}_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$$

$$\theta \leftarrow \theta + \alpha \gamma^t \hat{G}_t \nabla_{\theta} \ln \pi(u_t | x_t, \theta)$$

until end episode

end for

REINFORCE Algorithm

Input: a differentiable policy parameterization $\pi(u|x, \theta)$

Parameters: step size $\alpha \in (0, 1]$

Initialize: policy parameters θ

for each episode **do** Episodic algorithm! Complete full episode for one parameter update

 Generate an episode $x_0, u_0, r_1, \dots, x_{T-1}, u_{T-1}, r_T$

repeat for episode step $t = 0, 1, \dots, T$

$$\hat{G}_t = \sum_{k=t+1}^T \gamma^{k-t-1} r_k$$

 Episodic nature to obtain estimate of G

$$\theta \leftarrow \theta + \alpha \gamma^t \hat{G}_t \nabla_{\theta} \ln \pi(u_t | x_t, \theta)$$

until end episode

end for



Types of RL Control

By path to optimal solution

- **Off-policy** – find optimal policy regardless of the agent's motivation
- **On-policy** – improve the policy that is used to make decisions

By level of interaction with the process

- **Online** – learn by interacting with the process
- **Offline** – data collected in advance (Monte-Carlo methods)

By model knowledge

- **Model-free** – no p , only transition data (e.g. standard Q-Learning RL)
- **Model-based** – p is known (e.g. Dynamic Programming)
- **Model-learning** – estimate p from transition data

REINFORCE + baseline algorithm

$$\theta_{k+1} = \theta_k + \alpha(\hat{G}_k - b(x_k)) \nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$$

REINFORCE + baseline algorithm

$$\theta_{k+1} = \theta_k + \alpha(\hat{G}_k - b(x_k)) \nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$$

Baseline: We can introduce arbitrary baseline $b(x)$ without consequence*:

$$\nabla_{\theta} J(\theta, x_k) = \mathbb{E}_{\pi} \{ (G_k - b(x_k)) \nabla_{\theta} \pi(u_k | x_k, \theta) \}$$

Explanation: baseline *does not introduce bias* in gradient

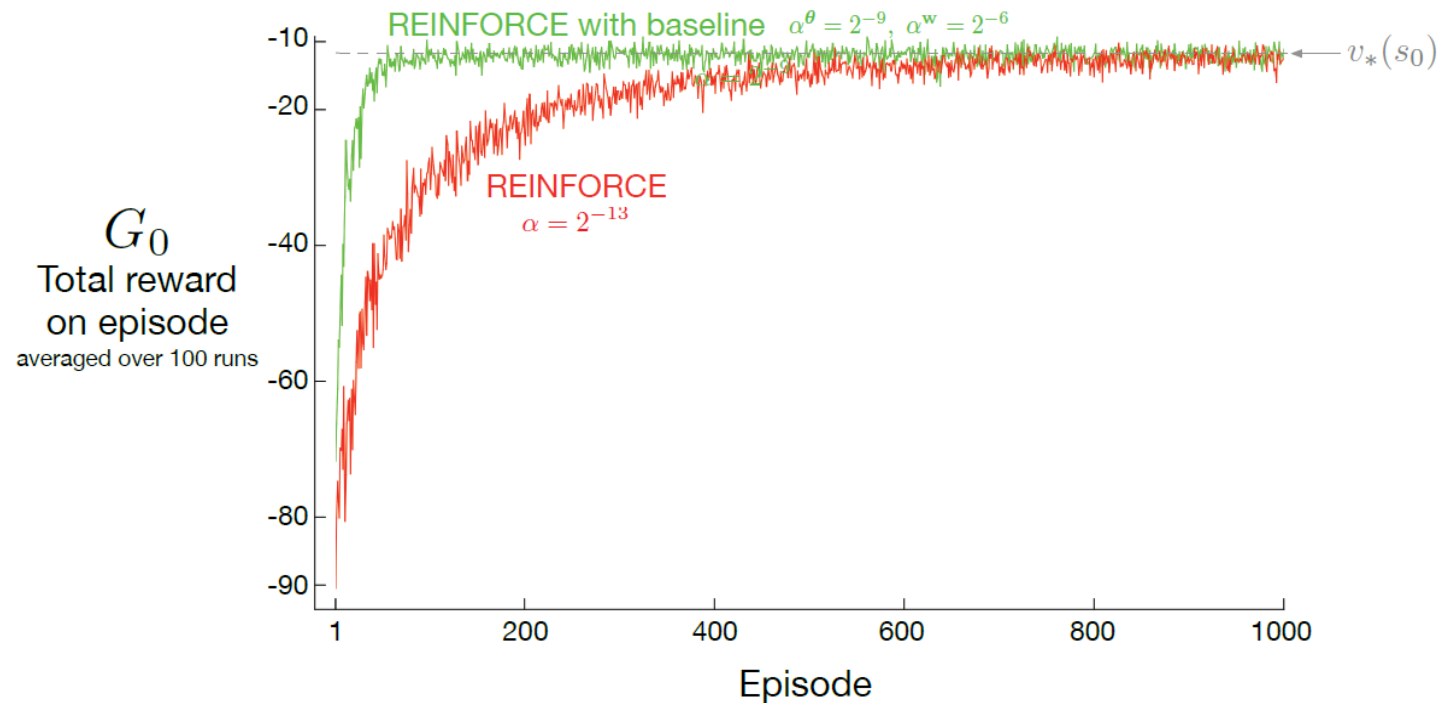
$$\sum_u b(x_k) \nabla_{\theta} \pi(u | x_k, \theta) = b(x_k) \nabla_{\theta} \sum_u \pi(u | x_k, \theta) = b(x_k) \nabla_{\theta} 1 = 0$$

But *reduces variance* of the gradient estimate in practice if

$$G_k \approx b(x_k)$$

* As long as b does not depend on u

REINFORCE + baseline algorithm



Actor-Critic Methods

Why Actor-Critic?

Problem: REINFORCE uses episode learning to estimate G_k

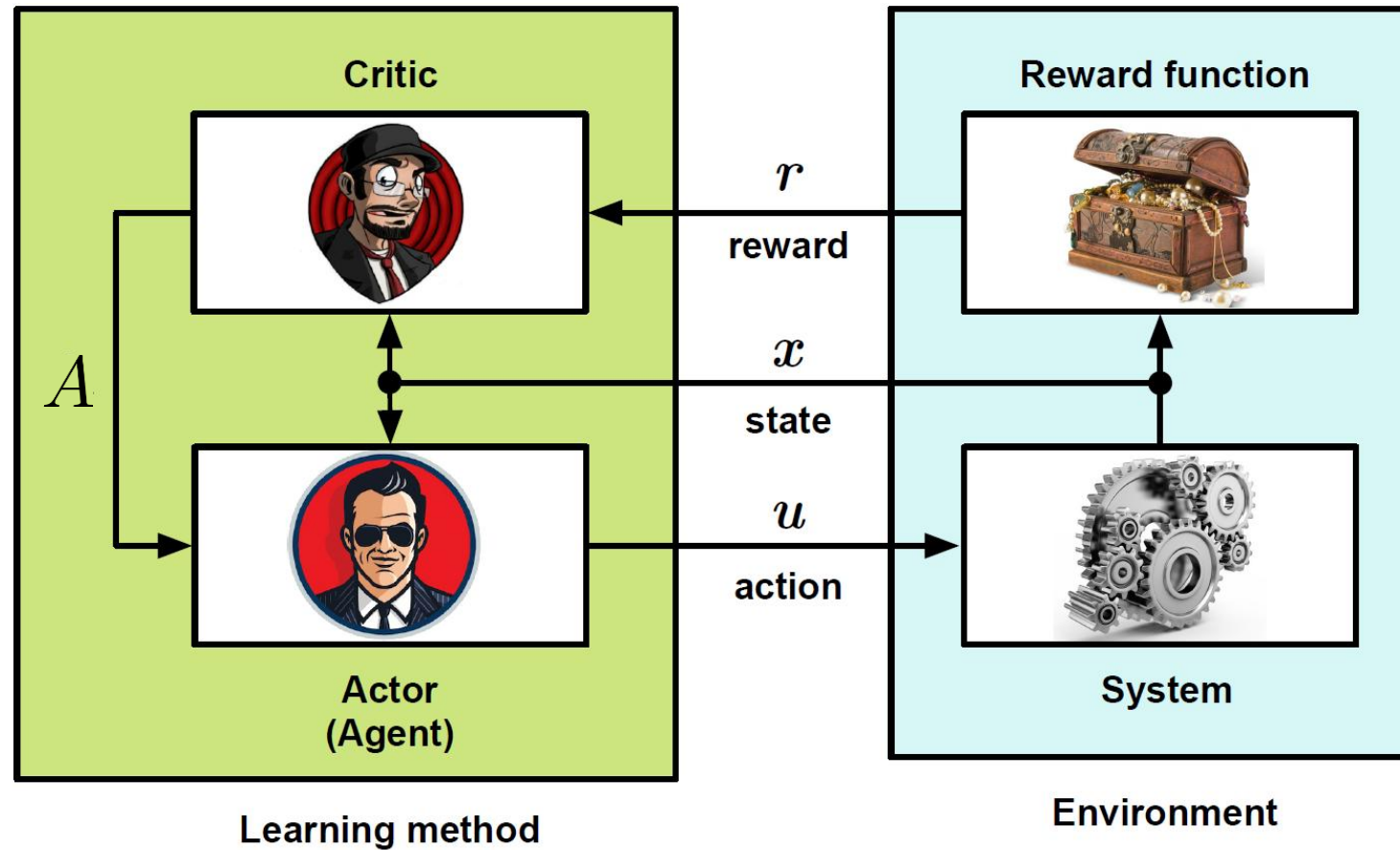
Idea: Introduce value function (cfr. Lecture 5 and 6)

$$\mathbb{E}_{\pi} \left\{ \sum_{\tau=0}^{\infty} r_{k+\tau+1} \mid x_k \right\} = \mathbb{E}_{\pi} \{ r_k + V_{\pi}(x_{k+1}) \}$$



Estimate using temporal difference
approach (Lecture 6)

Action-Critic Reinforcement Learning



Strategy: the **critic** aims to learn V_π while the **actor** tries to improve π

Actor-Critic Overview

Explicitly separated value function and policy

Critic = state value function $V_{\pi}(x)$

Actor = control policy $\pi(x)$

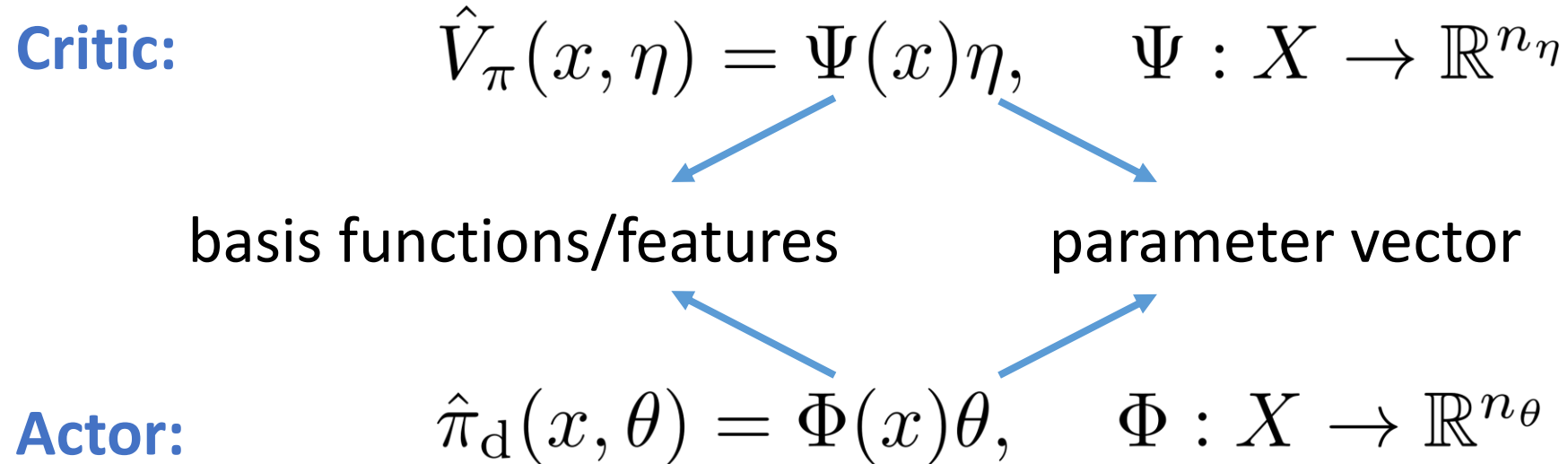
Problem setting:

Continuous synchronous learning

Continuous action and state space

Deterministic state transition and reward function

Actor-Critic Parameterization



Alternatively, they can be described via GP, DNN, etc.

Don't forget: *probabilistic policies* (softmax, Gaussian, ϵ -greedy)



On-Policy Advantage Based Critic

Objective Critic: learn $V_\pi(x)$ for the current policy π

Step 1: Use the Bellman equation for $V_\pi(x_k)$

$$\begin{aligned} V_\pi(x_k) &= \mathbb{E}_\pi \left\{ \rho(x_k, \pi(x_k, \theta_k)) + \gamma V_\pi(x_{k+1}) \mid x_k \right\} \\ &= \mathbb{E}_\pi \left\{ \rho(x_k, u_k) + \gamma V_\pi(x_{k+1}) \mid x_k \right\} \\ &\approx r_{k+1} + \gamma V_\pi(x_{k+1}) \end{aligned}$$



Sample



On-Policy Advantage Based Critic

Objective Critic: learn $V_\pi(x)$ for the current policy π

Step 1: Use the Bellman equation for $V_\pi(x_k)$

$$V_\pi(x_k) = \mathbb{E}_\pi \left\{ \rho(x_k, \pi(x_k, \theta_k)) + \gamma V_\pi(x_{k+1}) \mid x_k \right\} \approx r_{k+1} + \gamma V_\pi(x_{k+1})$$

Step 2: Compute *advantage* using temporal difference

$$A_k = Q_\pi(x_k, u_k) - V_\pi(x_k)$$

use the transition sample

$$A_k = \mathbb{E}_\pi \{ r_{k+1} + \gamma V_\pi(x_{k+1}) \mid x_k, u_k \} - V_\pi(x_k)$$

$$A_k = r_{k+1} + \gamma V_\pi(x_{k+1}) - V_\pi(x_k)$$

$$\hat{A}_k = r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \eta_k) - \hat{V}_\pi(x_k, \eta_k)$$

Bootstrap

On-Policy Advantage Based Critic



Step 3: Minimize prediction error of the current estimate $\hat{V}_\pi$

$$\text{cost: } L_{VF} = \sum_{x \in \hat{X}} (V_\pi(x) - \hat{V}_\pi(x))^2$$



set of observations (current sample, replay, entire batch, ...)

On-Policy Advantage Based Critic



Step 3: Minimize prediction error of the current estimate $\hat{V}_\pi$

$$\text{cost: } L_{VF} = \sum_{x \in \hat{X}} (V_\pi(x) - \hat{V}_\pi(x))^2$$

↪ set of observations (current sample, replay, entire batch, ...)

Take $\hat{X} = \{x_k\}$, SGD gives:

$$\eta_{k+1} = \eta_k + \alpha_c \hat{A}_k \nabla_\eta \hat{V}_\pi(x_k, \eta_k)$$

↪ learning rate



On-Policy Advantage Based Critic

Step 3: Minimize prediction error of the current estimate $\hat{V}_\pi$

$$\text{cost: } L_{VF} = \sum_{x \in \hat{X}} (V_\pi(x) - \hat{V}_\pi(x))^2$$

↪ set of observations (current sample, replay, entire batch, ...)

Take $\hat{X} = \{x_k\}$, SGD gives:

$$\eta_{k+1} = \eta_k + \alpha_c \hat{A}_k \nabla_\eta \hat{V}_\pi(x_k, \eta_k)$$

↪ learning rate

See also the prediction problem in Lecture 7!

On-Policy Advantage Based Critic



$$\eta_{k+1} = \eta_k + \alpha_c \hat{A}_k \nabla_{\eta} \hat{V}_{\pi}(x_k, \eta_k)$$

$$\hat{A}_k > 0 \quad r_{k+1} + \gamma \hat{V}_{\pi}(x_{k+1}, \eta_k) > \hat{V}_{\pi}(x_k, \eta_k)$$

Old estimate is too low $\rightarrow$ increase it (too negative criticism in the past)

$$\hat{A}_k < 0 \quad r_{k+1} + \gamma \hat{V}_{\pi}(x_{k+1}, \eta_k) < \hat{V}_{\pi}(x_k, \eta_k)$$

Old estimate is too high $\rightarrow$ decrease it (too enthusiastic opinion in the past)



On-Policy Advantage Based Actor

Remember REINFORCE + baseline:

$$\theta_{k+1} = \theta_k + \alpha(\hat{G}_k - b(x_k))\nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$$

Step 1 Use Bellman equation + transition sample + critic-based estimate of V_{π}

$$\mathbb{E}_{\pi}\{G_k | x_k\} = \mathbb{E}_{\pi}\{r_{k+1} + V_{\pi}(x_{k+1})\} \Rightarrow \hat{G}_k = r_{k+1} + \hat{V}_{\pi}(x_{k+1})$$

Step 2 Use the baseline $b(x_k) = \hat{V}_{\pi}(x_k)$

$$\hat{G}_k - b(x_k) = r_{k+1} + \hat{V}_{\pi}(x_{k+1}) - \hat{V}_{\pi}(x_k) = \hat{A}_k$$



Advantage



On-Policy Advantage Based Actor

Step 3 Achieve maximization of $J(\theta, x_k)$ via SGA

$$\theta_{k+1} = \theta_k + \alpha_a \hat{A}_k \nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$$

learning rate



Probabilistic policy

Remember our goal was to maximize the objective



On-Policy Advantage Based Actor

Step 4 Gradient calculation $\nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$

Example: choose Gaussian policy $\pi_{\theta}(x, u) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(u - f_{\theta}(x))^2}{2\sigma^2}\right)$

$$\begin{aligned}\nabla_{\theta} \ln \pi(u_k | x_k, \theta) &= \nabla_{\theta} \ln \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(u_k - f_{\theta}(x_k))^2}{2\sigma^2}\right) \\ &= \nabla_{\theta} \frac{(u_k - f_{\theta}(x_k))^2}{-2\sigma^2} \\ &= \frac{1}{\sigma^2} (u_k - f_{\theta}(x_k)) \nabla_{\theta} f_{\theta}(x_k)\end{aligned}$$



On-Policy Advantage Based Actor

$$\theta_{k+1} = \theta_k + \alpha_a \hat{A}_k \nabla_{\theta} \ln \pi(u_k | x_k, \theta_k)$$

$$\hat{A}_k > 0 \quad r_{k+1} + \gamma \hat{V}_{\pi}(x_{k+1}, \eta_k) > \hat{V}_{\pi}(x_k, \eta_k)$$

Action u_k has unforeseen advantage (exploit it more in the future)

$$\hat{A}_k < 0 \quad r_{k+1} + \gamma \hat{V}_{\pi}(x_{k+1}, \eta_k) < \hat{V}_{\pi}(x_k, \eta_k)$$

Action u_k has unforeseen disadvantage (avoid it in the future)

Advantage Actor-Critic (A2C) Algorithm

Input: a differentiable parameterization for $\pi(u|x, \theta)$ and $V_\pi(x, \eta)$

Parameters: step size $\alpha_a, \alpha_c \in (0, 1]$

Initialize: actor and critic parameters θ, η

for each episode **do**

 initialise x_0

repeat for each step k

 obtain u_k based on policy π

 apply u_k , measure x_{k+1} , receive r_{k+1}

$\hat{A}_k = r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \eta) - \hat{V}_\pi(x_k, \eta)$

$\eta \leftarrow \eta + \alpha_c \hat{A}_k \nabla_\eta \hat{V}_\pi(x_k, \eta)$

$\theta \leftarrow \theta + \alpha_a \hat{A}_k \nabla_\theta \ln \pi(u_k | x_k, \theta)$

until terminal state

end for

advantage

critic update - minimization

actor update - maximization

Note: you can also run multiple actors in parallel

Exploration-Exploitation: Entropy Regularizer

Maximize Value Function + Entropy Regularization

$$J_S(\theta, x) = J(\theta, x) + \lambda S(\pi(\cdot|x, \theta))$$

Entropy: Measure of uncertainty / information

$$S(p) = - \sum_x p(x) \ln(p(x))$$

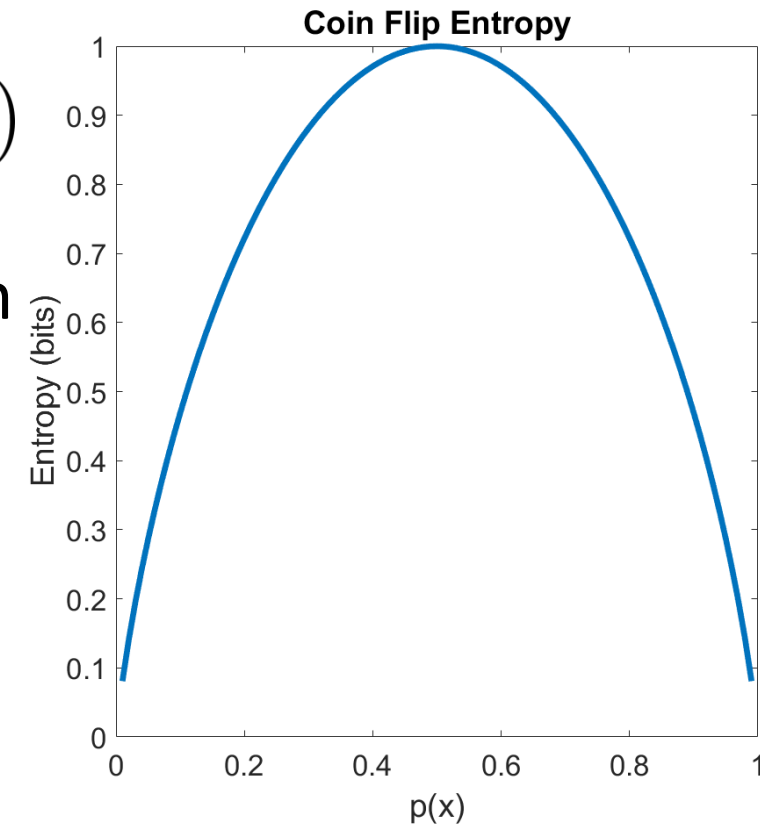
Exploration-Exploitation: Entropy Regularizer

Maximize Value Function + Entropy Regularization

$$J_S(\theta, x) = J(\theta, x) + \lambda S(\pi(\cdot|x, \theta))$$

Entropy: Measure of uncertainty / information

$$S(p) = - \sum_x p(x) \ln(p(x))$$



Exploration-Exploitation: Entropy Regularizer

Maximize Value Function + Entropy Regularization

$$J_S(\theta, x) = J(\theta, x) + \lambda S(\pi(\cdot|x, \theta))$$

Entropy: Measure of uncertainty / information

$$S(p) = - \sum_x p(x) \ln(p(x))$$

increases with
uncertainty



Exploration-Exploitation Trade-Off



Types of RL Control

By path to optimal solution

- **Off-policy** – find optimal policy regardless of the agent's motivation
- **On-policy** – improve the policy that is used to make decisions

By level of interaction with the process

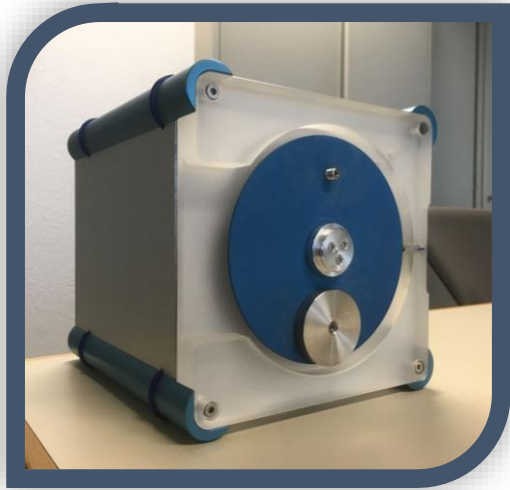
- **Online** – learn by interacting with the process
- **Offline** – data collected in advance (Monte-Carlo methods)

By model knowledge

- **Model-free** – no p , only transition data (e.g. standard Q-Learning RL)
- **Model-based** – p is known (e.g. Dynamic Programming)
- **Model-learning** – estimate p from transition data

Examples

Unbalanced Disc



- State: $x = (\alpha \quad \dot{\alpha})$ (angle and velocity)
 - Input: $u = \text{voltage}$
 - Reward: $r_{k+1} = \rho(x_k, u_k) = -x_k^\top \begin{bmatrix} 5 & 0 \\ 0 & 0.1 \end{bmatrix} x_k - u_k^\top u_k$
 - Discount: $\gamma = 0.98$
-
- Objective: stabilize top position (swing up)
 - Insufficient actuation (needs to swing back & forth)

Unbalanced Disc

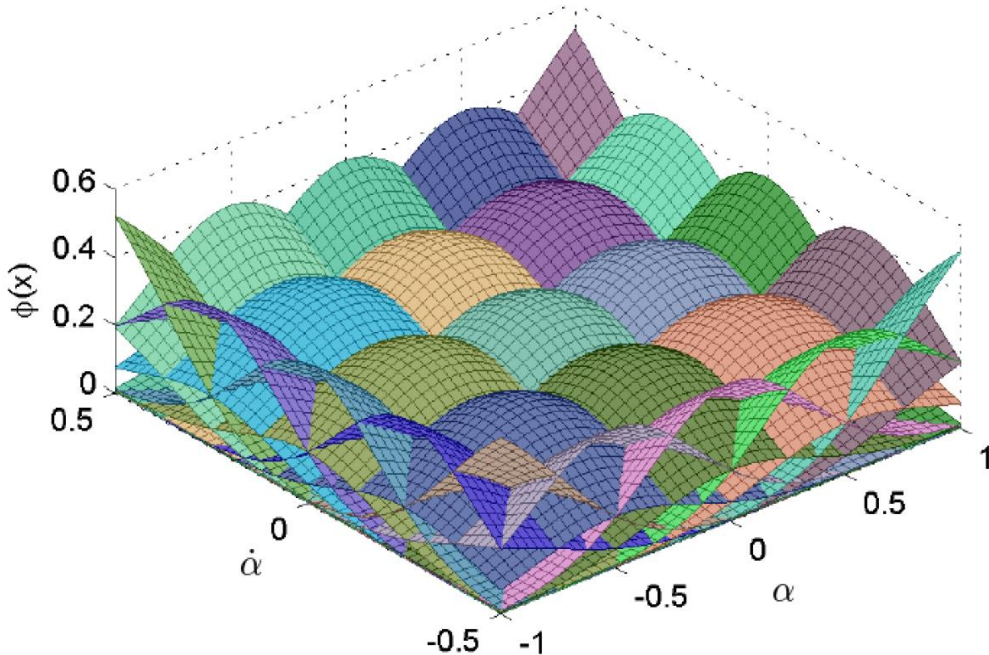
Parameterization

$$\text{Critic: } \hat{V}_\pi(x, \eta) = \Psi(x)\eta$$

$$\text{Actor: } \hat{\pi}_d(x, \theta) = \Phi(x)\theta$$

deterministic part only

Basis functions are given by normalized RBF with equidistant gridding



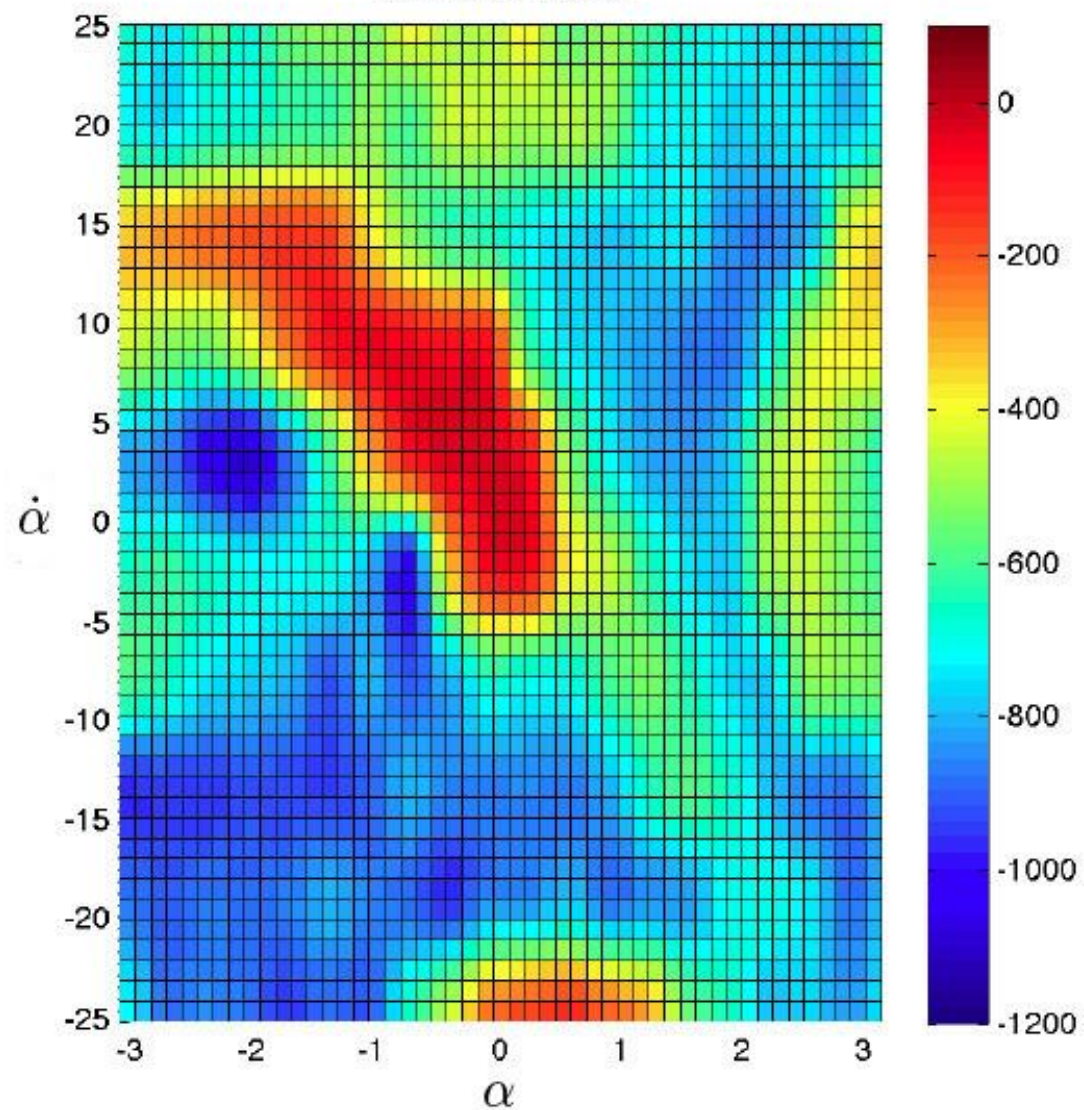
$$\phi_i(x) = \frac{\phi_i(x)}{\sum_{j=1}^{n_\theta} \phi_j(x)}$$

$$\phi_i(x) = \exp\left(-\frac{\|x - c_i\|_2^2}{2\sigma^2}\right)$$

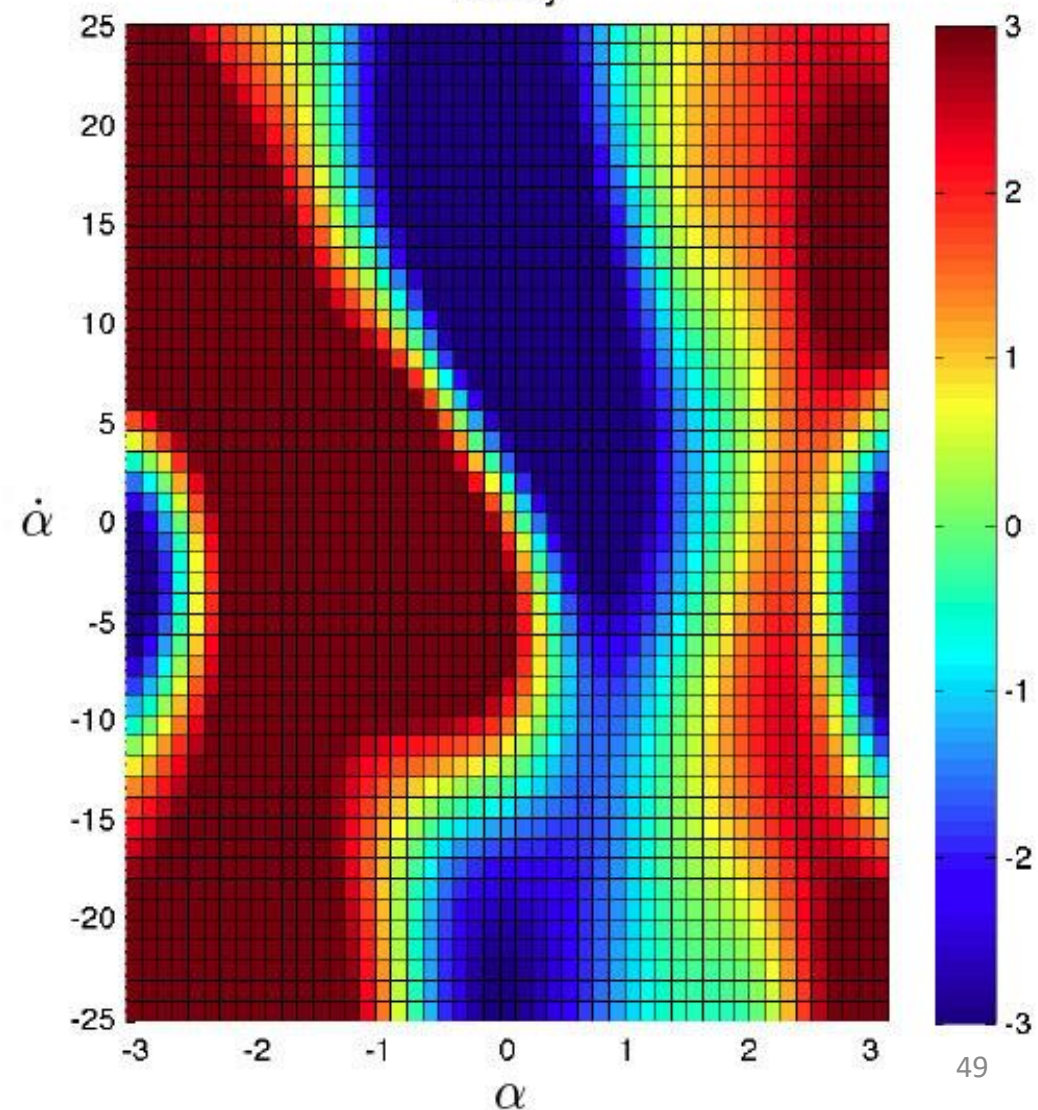


Unbalanced Disc

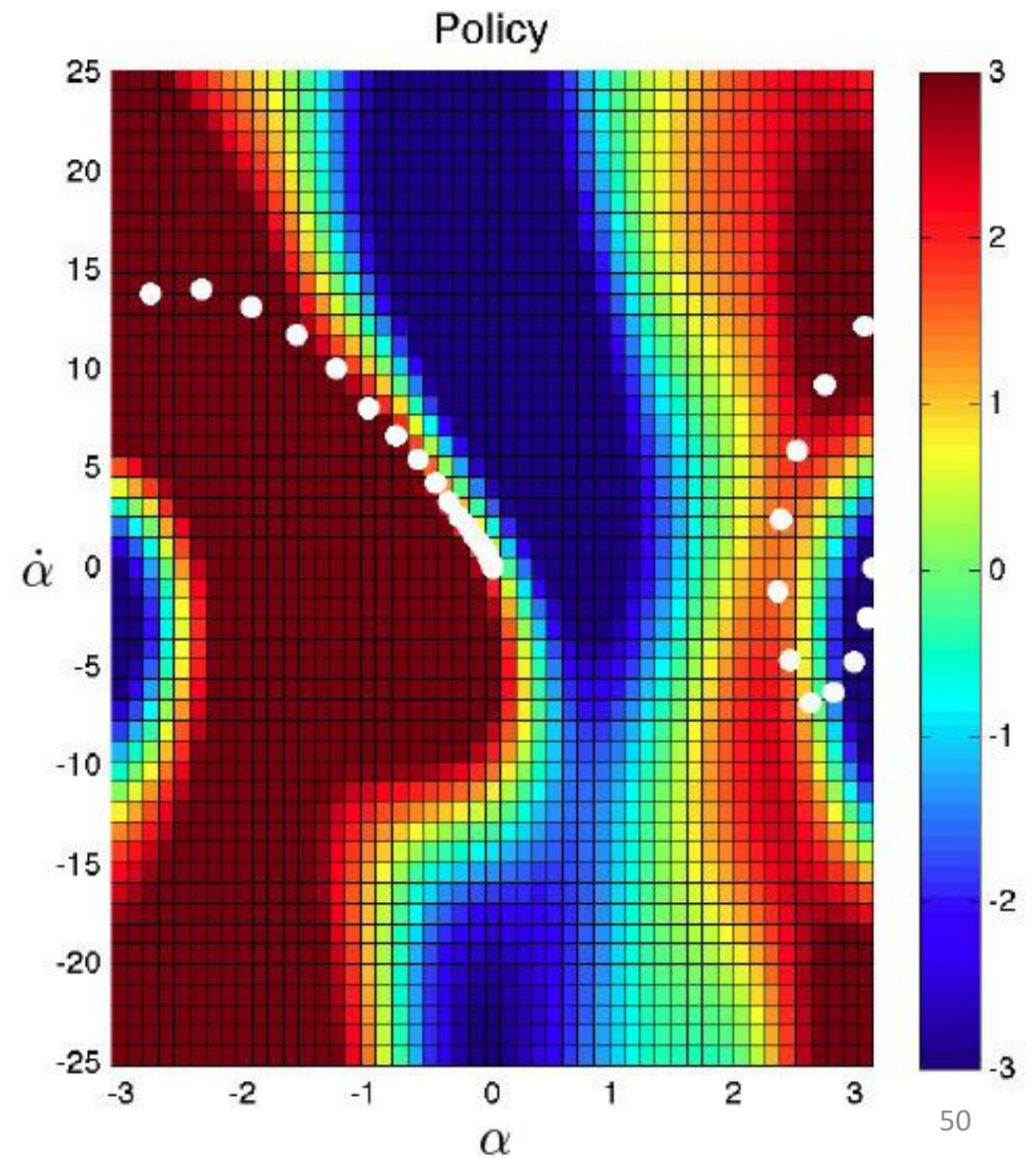
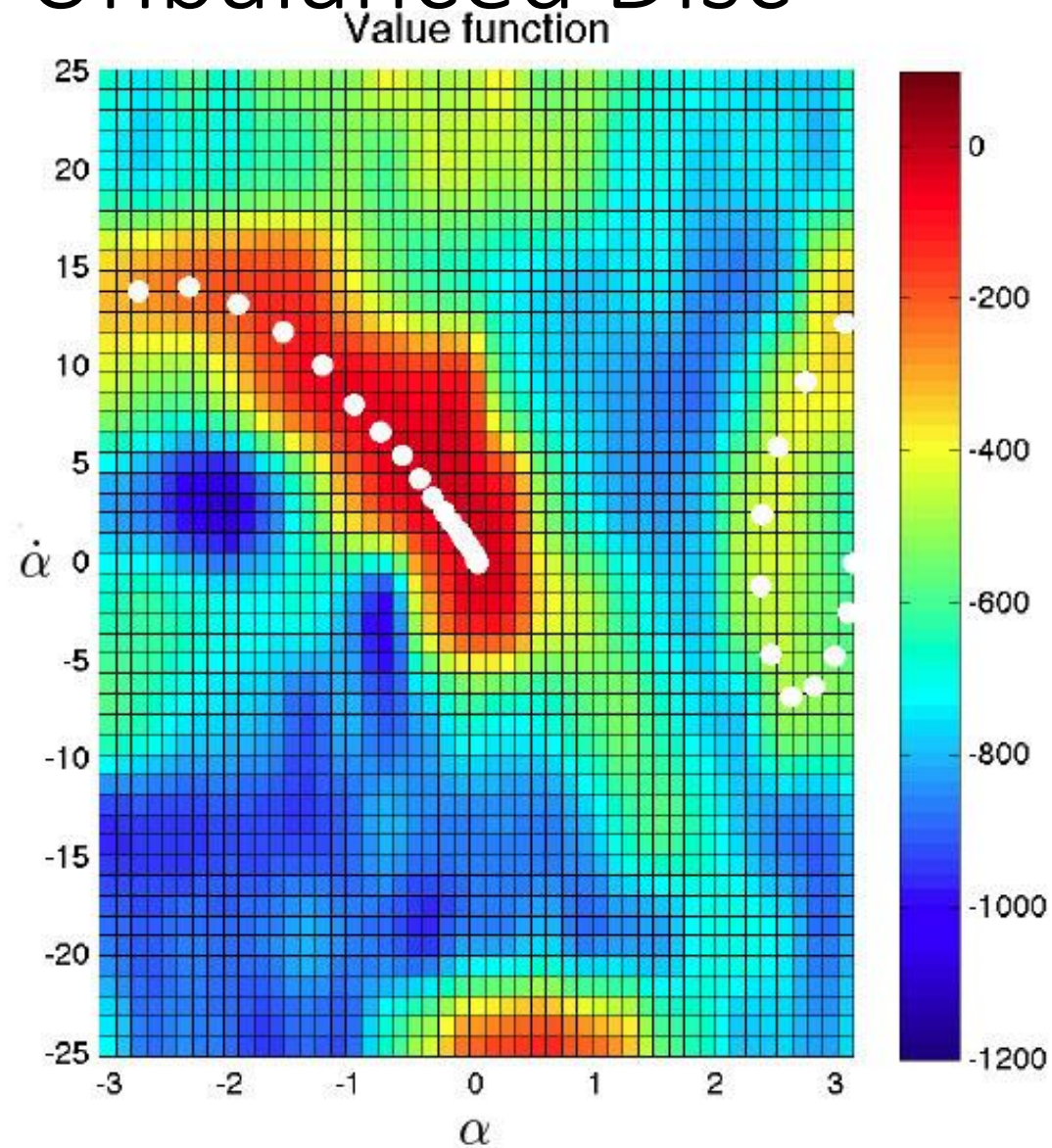
Value function



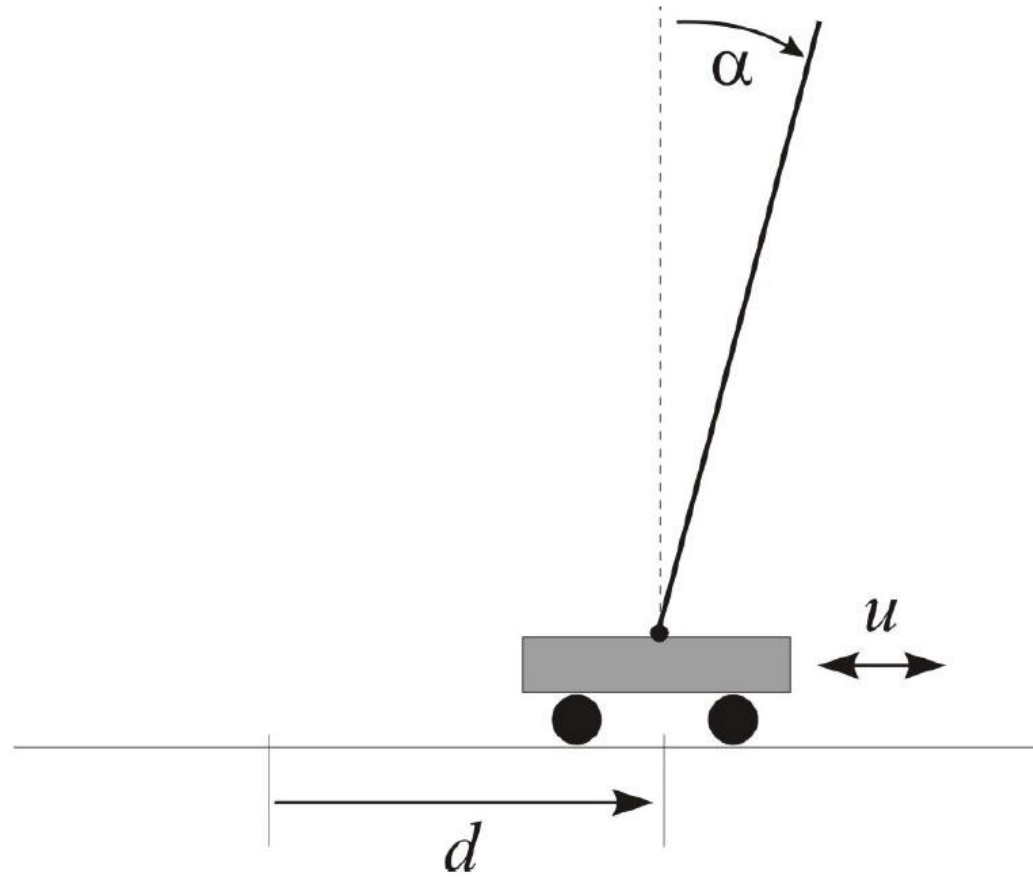
Policy



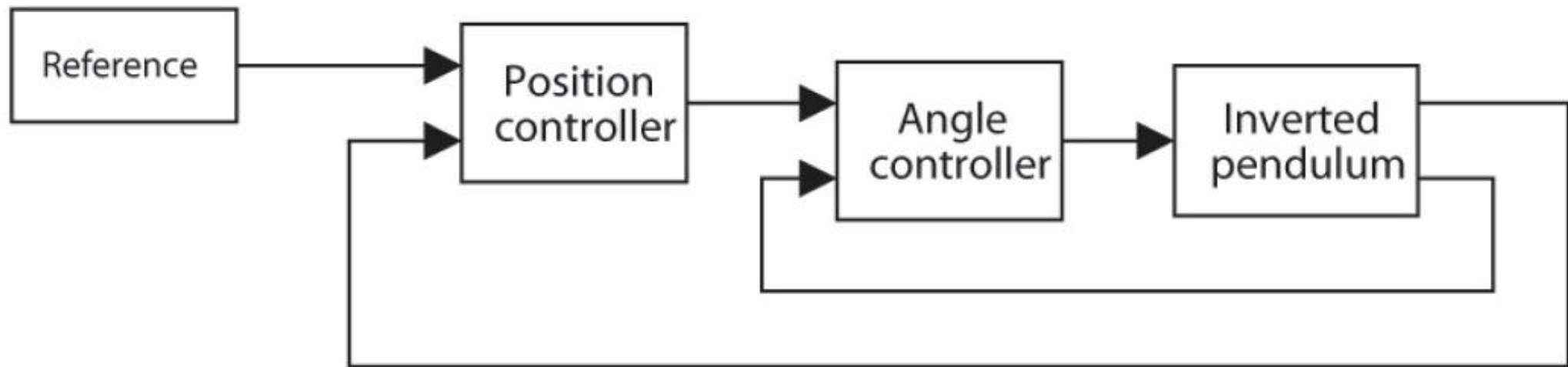
Unbalanced Disc



Cart with Pendulum

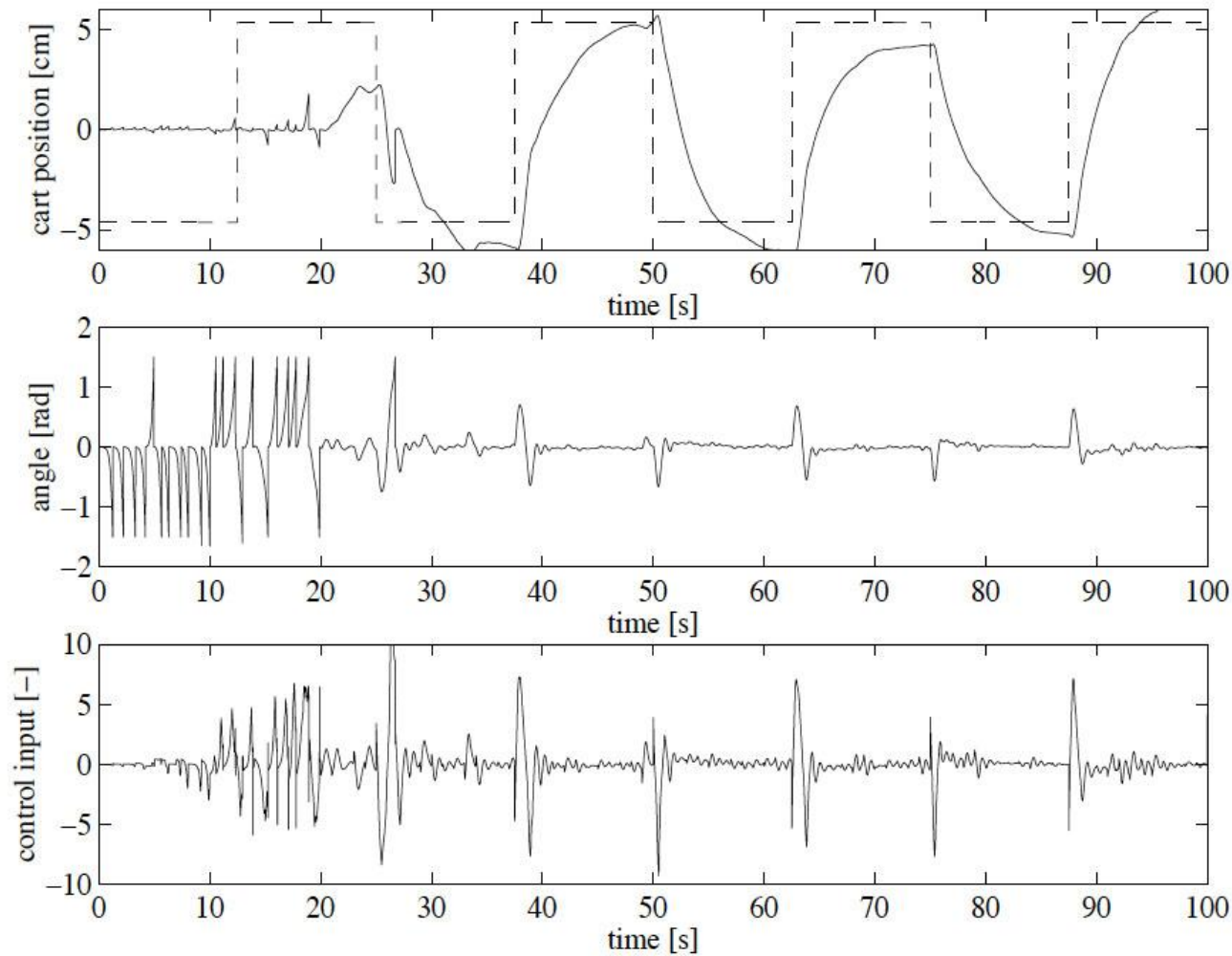


Cart with Pendulum



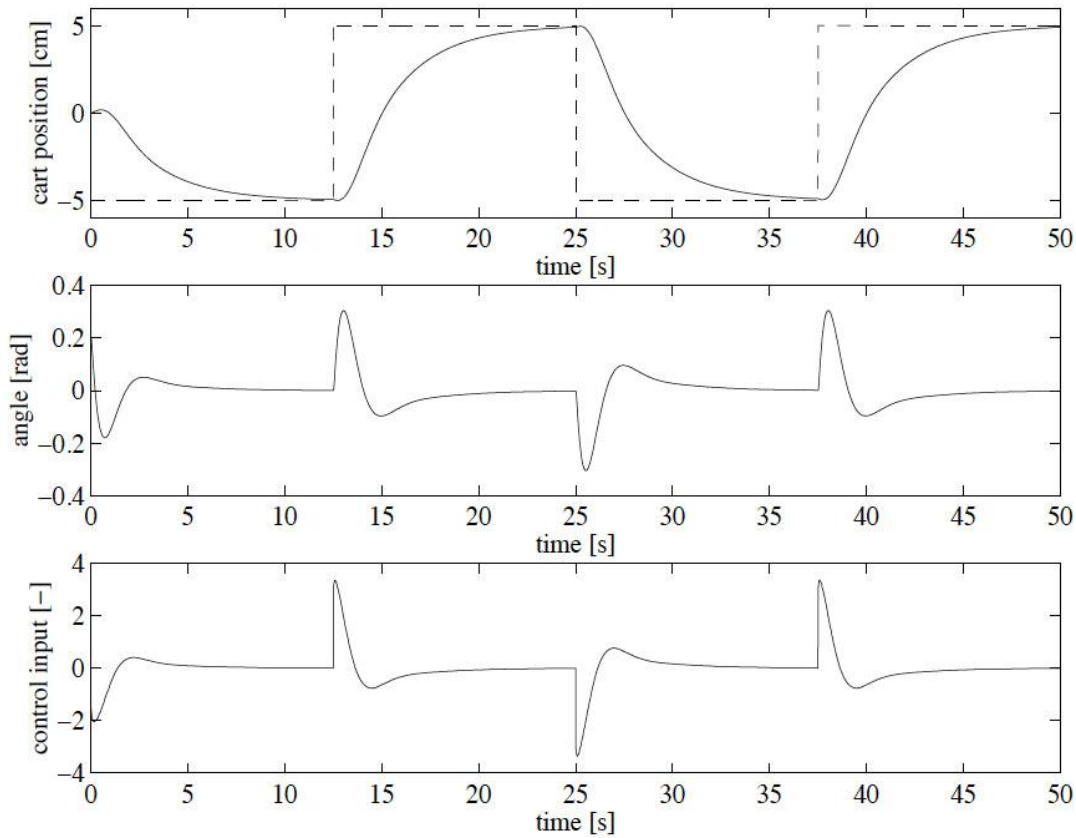
Hierarchical control loop. The position controller is fixed to be a pre-tuned PD controller. The objective is to learn the angle controller in closed loop.

Cart with Pendulum: Learning

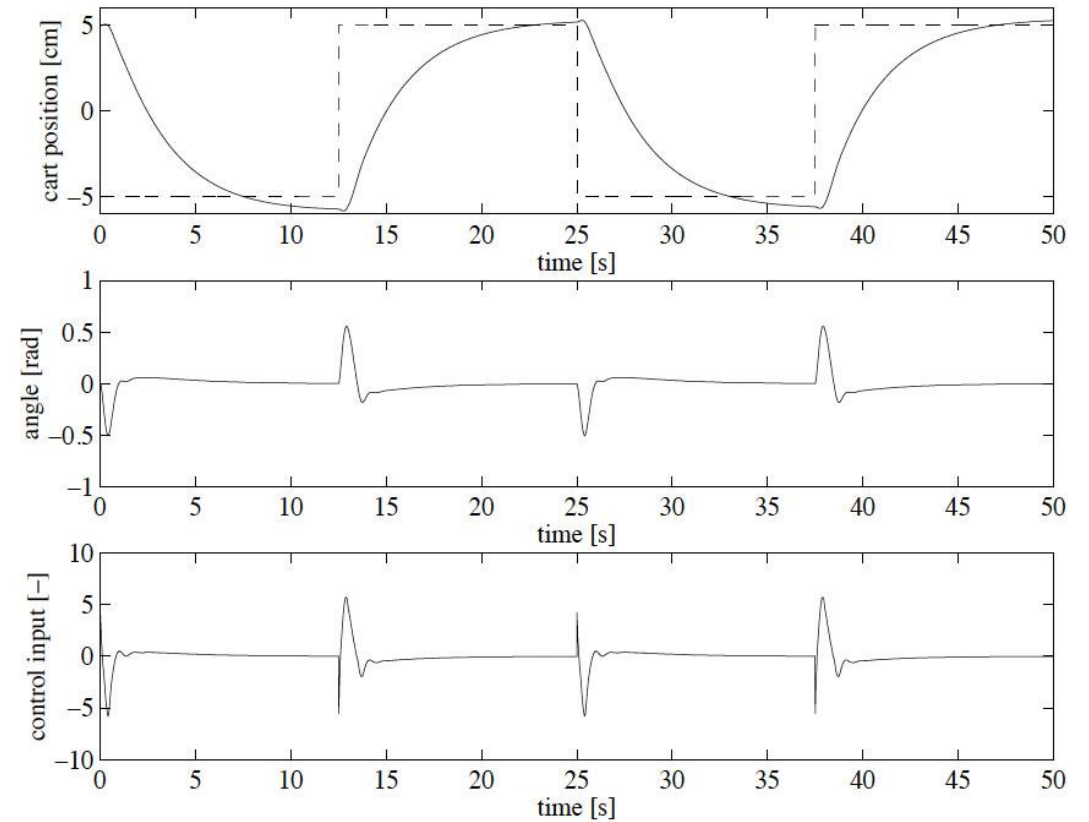


Performance during learning

Cart with Pendulum: RL vs Classical

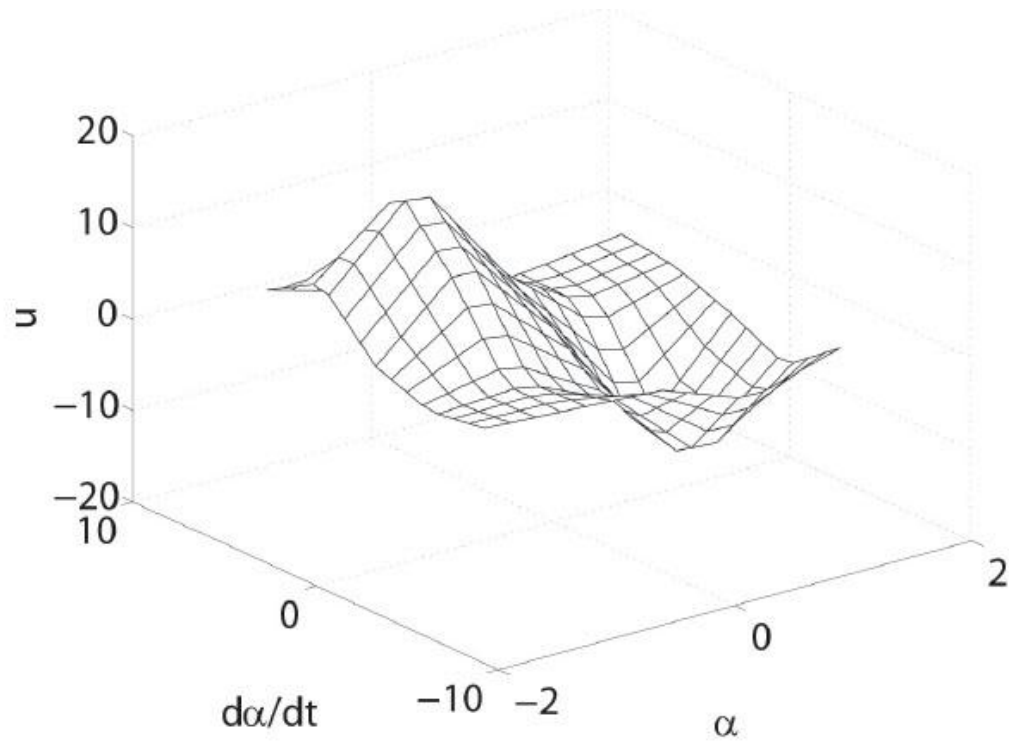


Classical Control Design

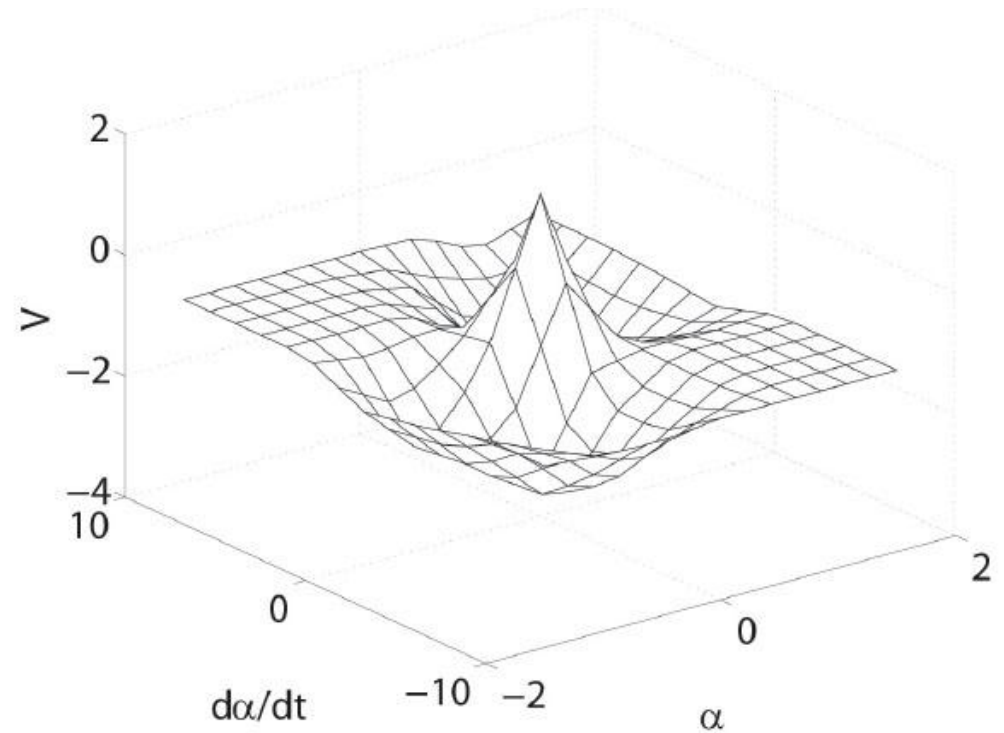


Reinforcement Learning

Cart with Pendulum: Actor and Critic



Actor

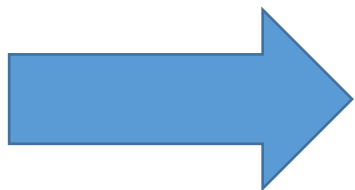


Critic

Proximal Policy Optimization

$$J_S(\theta, x) = J(\theta, x) + \lambda S(\pi(\cdot|x, \theta))$$

Changes: Longer optimization on data generated by old policy
 Probability Ratio instead of Policy Probability
 Clipping *or* KL Divergence



Increased Stability
No Policy Run-Off due to Large Updates
Observed Improved Sample Efficiency

Proximal Policy Optimization: Efficiency

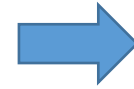
1. Data generated by old policy
2. Advantage computed on that data



We get policy run off if we train new policy too long on data from old policy

Proximal Policy Optimization: Efficiency

1. Data generated by old policy
2. Advantage computed on that data



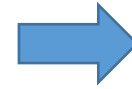
We get policy run off if we train new policy too long on data from old policy

But! Importance Sampling: $\mathbb{E}_{x \sim p} \{f(x)\} = E_{x \sim q} \left\{ f(x) \frac{p(x)}{q(x)} \right\}$

Will allow us to use samples from old policy as if they came from the current policy distribution by applying correct weighting

Proximal Policy Optimization: Efficiency

1. Data generated by old policy
2. Advantage computed on that data



We get policy run off if we train new policy too long on data from old policy
→ wrong gradient information

But! Importance Sampling: $\mathbb{E}_{x \sim p} \{f(x)\} = \mathbb{E}_{x \sim q} \left\{ f(x) \frac{p(x)}{q(x)} \right\}$

AC

$$J(\theta, x) = \mathbb{E}_k \left\{ \ln \pi(u_k | x_k, \theta) \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$



Introduce Policy Ratio, as we have samples from old policy

towards
PPO

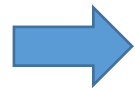
$$J(\theta, x) = \mathbb{E}_k \left\{ \frac{\pi(u_k | x_k, \theta)}{\pi(u_k | x_k, \theta_{old})} \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$

Proximal Policy Optimization: Stability

$$J(\theta, x) = \mathbb{E}_k \left\{ \frac{\pi(u_k | x_k, \theta)}{\pi(u_k | x_k, \theta_{old})} \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$

Optimizing this results in excessively large policy updates

Limit the update range

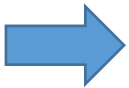


Only update where we have a large trust
in our updates

Proximal Policy Optimization: Stability

$$J(\theta, x) = \mathbb{E}_k \left\{ \frac{\pi(u_k | x_k, \theta)}{\pi(u_k | x_k, \theta_{old})} \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$

Optimizing this results in excessively large policy updates

Limit the update range  Only update where we have a large trust in our updates

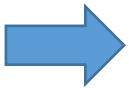
Option 1: Use KL-divergence to measure difference between action distributions and add as a penalty with variable weight in the cost

Option 2: Limit the policy ratio to a given range $[1-\epsilon, 1+\epsilon]$

Proximal Policy Optimization: Stability

$$J(\theta, x) = \mathbb{E}_k \left\{ \frac{\pi(u_k | x_k, \theta)}{\pi(u_k | x_k, \theta_{old})} \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$

Optimizing this results in excessively large policy updates

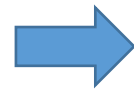
Limit the update range  Only update where we have a large trust in our updates

Option 1: Use KL-divergence to measure difference between action distributions and add as a penalty with variable weight in the cost

Option 2: Limit the policy ratio to a given range $[1-\epsilon, 1+\epsilon]$

Proximal Policy Optimization: Stability

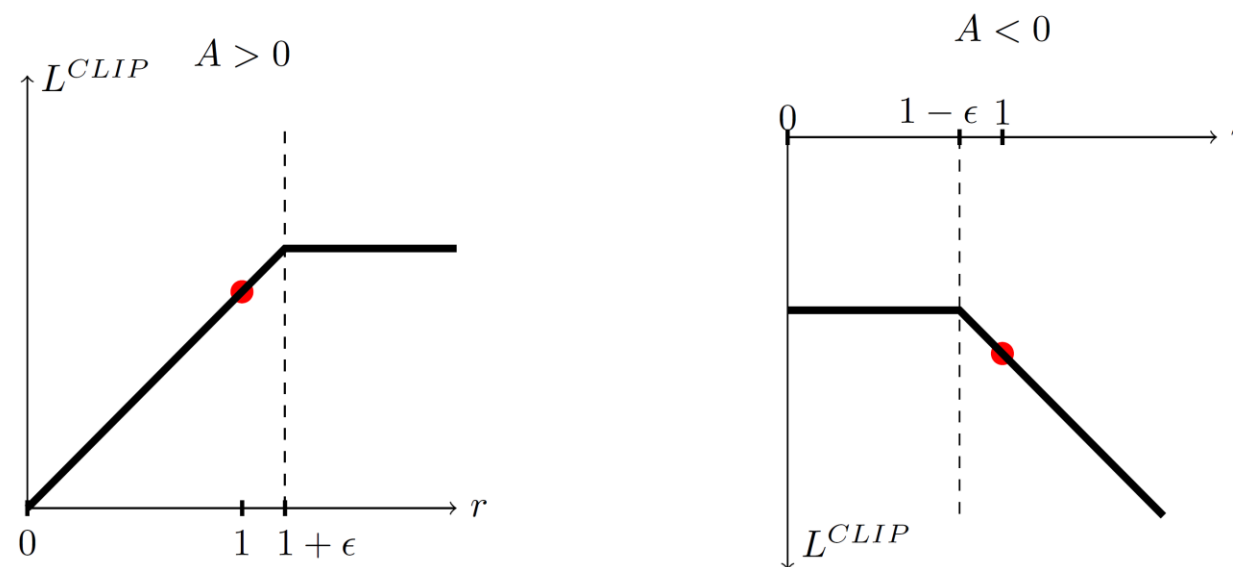
Limit the update range



Only update where we have a large trust in our updates

Option 2: Limit the policy ratio to a given range $[1-\epsilon, 1+\epsilon]$

Clipping:



Source: J. Schulman et al., Proximal Policy Optimization Algorithms, arXiv:1707.06347

Figure 1: Plots showing one term (i.e., a single timestep) of the surrogate function L^{CLIP} as a function of the probability ratio r , for positive advantages (left) and negative advantages (right). The red circle on each plot shows the starting point for the optimization, i.e., $r = 1$. Note that L^{CLIP} sums many of these terms.

Proximal Policy Optimization: Stability

$$J(\theta, x) = \mathbb{E}_k \left\{ \frac{\pi(u_k | x_k, \theta)}{\pi(u_k | x_k, \theta_{old})} \hat{A}_k \right\} + \lambda S(\pi(\cdot | x, \theta))$$



Introduce Clipping + Min

$$J_{\text{clip}}(\theta, x) = \mathbb{E}_k \left\{ \min \left(r_k(\theta) \hat{A}_k, \text{clip} (r_k(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_k \right) \right\}$$



$$J(\theta, x) = J_{\text{clip}}(\theta, x) + \lambda S(\pi(\cdot | x, \theta))$$

Proximal Policy Optimization: Cost

$$J(\theta, x) = J_{\text{clip}}(\theta, x) + \lambda_1 S(\pi(\cdot|x, \theta)) - \lambda_2 L_{VF}(\theta, x)$$

 Entropy

$$J_{\text{clip}}(\theta, x) = \mathbb{E}_k \left\{ \min \left(r_k(\theta) \hat{A}_k, \text{clip} (r_k(\theta), 1 - \epsilon, 1 + \epsilon) \right) \right\}$$

 Surrogate Objective Function

$$L_{VF}(\theta) = \sum_{x \in \hat{X}} (V_{\pi}(x) - \hat{V}_{\pi}(x, \theta))^2 \quad \text{Critic Objective Function}$$

Proximal Policy Optimization: Cost

Initialize: actor and critic parameters θ

for each episode **do**

 initialise x_0

 run policy $\pi_{\theta_{old}}$ in environment for T timesteps

 compute advantage estimates $\hat{A}_1, \dots, \hat{A}_T$

 Maximize $J(\theta, x)$ wrt θ for K epochs and chosen minibatch size

$\theta_{old} \leftarrow \theta$

end for

Note: you can also run multiple actors in parallel

Actor-Critic Properties

Proof of convergence under various assumptions

Improved stability

Probabilistic in nature

On-policy and off-policy versions

Tabular and continuous formulation

Many tricks to improve performance (reward normalization, ...)

Updates can be synchronous, asynchronous or episodic

Many versions (A2C, A3C, SAC, PPO, ...)

Overview

Introduced policy gradient reinforcement learning

REINFORCE

Actor-Critic (A2C)

PPO

Can work probabilistic, handle continuous state *and* action space

Fundamental dilemma remains:

- Efficiency of DP: Models make it possible to plan and synthesize policy, otherwise we only rely on experience. Experiments are costly and risky.
- Efficiency of RL: Experiments allow to explore and improve exploitation on the long run. Models are inherently uncertain.
- How to have a working marriage of DP and RL?